

## Searching

Search as the name suggests, is an operation of finding an item from the given collection of items. In other words we check if a given element (called key) occurs in the array.

– Example: array of student records; rollno can be the key.

## Linear Search

Linear search or sequential search is a method for finding a particular value in a list that checks each element in sequence until the desired element is found or the list is exhausted. The list need not be ordered. It is the simplest search algorithm; it is a special case of brute-force search.

Basic idea:

- *Start at the beginning of the array.*
- *Inspect elements one by one to see if it matches the key.*

Time complexity:

- It is a measure of how long an algorithm takes to run.
- If there are  $n$  elements in the array:
  - Best case:  
match found in first element (1 search operation)
  - Worst case:  
no match found, or match found in the last element ( $n$  search operations)
  - Average case:  
 $(n + 1) / 2$  search operations

Program:

```
/*  
* C program to input N numbers and store them in an array.  
* Do a linear search for a given key and report success or failure.  
*/  
  
#include <stdio.h>  
  
void main()  
{  
    int array[10];  
    int i, num, keynum, found = 0;  
    printf("Enter the value of num \n");  
    scanf("%d", &num);  
    printf("Enter the elements one by one \n");  
    for (i = 0; i < num; i++)  
    {  
        scanf("%d", &array[i]);  
    }  
    printf("Input array is \n");  
    for (i = 0; i < num; i++)  
    {  
        printf("%dn", array[i]);  
    }  
    printf("Enter the element to be searched \n");  
    scanf("%d", &keynum);
```

```

/* Linear search begins */
for (i = 0; i < num ; i++)
{
    if (keynum == array[i] )
    {
        found = 1;
        break;
    }
}
if (found == 1)
    printf("Element is present in the array\n");
else
    printf("Element is not present in the array\n");
}

```

A small modification of the same program that returns the position of the element (key) if found.

```

/*
* C program to input N numbers and store them in an array.
* Do a linear search for a given key and report success or failure also return the position of
the element if found.
*/
#include <stdio.h>

void main()
{
    int array[10];
    int i, num, keynum, pos, found = 0;

```

```

printf("Enter the value of num \n");

scanf("%d", &num);

printf("Enter the elements one by one \n");

for (i = 0; i < num; i++)
{
    scanf("%d", &array[i]);
}

printf("Input array is \n");

for (i = 0; i < num; i++)
{
    printf("%d\n", array[i]);
}

printf("Enter the element to be searched \n");

scanf("%d", &keynum);

/* Linear search begins */

for (i = 0; i < num ; i++)
{ if (keynum == array[i] )
    {
        found = 1;

        pos = i+1;

        break;
    } }

if (found == 1)

    printf("Element is present in the array at position %d\n", pos);

else

    printf("Element is not present in the array\n");

}

```

## Binary Search

A binary search or half-interval search algorithm finds the position of a target value within a sorted array. The binary search algorithm can be classified as a dichotomic divide-and-conquer search algorithm executes in logarithmic time.

### Basic Concept:

- Binary search works if the array is sorted.
  - Look for the target in the middle.
  - If you don't find it, you can ignore half of the array, and repeat the process with the other half.
- In every step, we reduce the number of elements to search in by half.

### Algorithm:

First, the list has to be sorted in increasing order.

It starts with the middle element of the list.

1. If the middle element of the list is equal to the 'input key' then we have found the position the specified value.
2. Else if the 'input key' is greater than the middle element then the 'input key' has to be present in the last half of the list.
3. Or if the 'input key' is lesser than the middle element then the 'input key' has to be present in the first half of the list.

Hence, the search list gets reduced by half after each iteration.

[low, high] denotes the range in which element has to be present and [mid] denotes the middle element.

Initially low = 0, high = number\_of\_elements and mid = floor((low+high)/2).

In every iteration we reduce the range by doing the following until low is less than or equal to high (meaning elements are left in the array) or the position of the 'input key' has been found.

(i) If the middle element (mid) is less than key then key has to present in range

[mid+1, high], so low=mid+1, high remains unchanged and mid is adjusted accordingly

(ii) If middle element is the key, then we are done.

(iii) If the middle element is greater than key then key has to be present in the range [low,mid-1], so high=mid-1, low remains unchanged and mid is adjusted accordingly.

#### Properties:

1. Best Case performance – The middle element is equal to the ‘input key’  $O(1)$ .
2. Worst Case performance - The ‘input key’ is not present in the list  $O(\log n)$ .
3. Average Case performance – The ‘input key’ is present, but it’s not the middle element  $O(\log n)$ .
4. The List should be sorted for using Binary Search Algorithm.
5. It is faster than Linear Search algorithm, and its performance increases in comparison to linear search as N grows.

To trace the algorithm with an example,

**Refer to Attachment: Arrays\_Binary\_Search\_Algorithm.pdf**

#### Program:

```
#include<stdio.h>
```

```
/* Logic: [low, high] denotes the range in which element has to be present.
```

```
Initially low = 0, high = number_of_elements which covers entire range.
```

```
In every step we reduce the range by doing the following
```

```
(i) If the middle element (mid) is less than key then key has to present in range [mid+1 , high]
```

```
(ii) If middle element is the key, then we are done.
```

```
(iii) If the middle element is greater than key then key has to be present in the range[low,mid-1]
```

```
*/
```

```
int main()
```

```
{
```

```
    int number_of_elements;
```

```

scanf("%d",&number_of_elements);

int array[number_of_elements];

int iter;

/*Input has to be sorted in non decreasing order */

for(iter = 0;iter < number_of_elements;iter++)
{
    scanf("%d",&array[iter]);
}

/* Check if the input array is sorted */

for(iter = 1;iter < number_of_elements;iter++)
{
    if(array[iter] < array[iter - 1])
    {
        printf("Given input is not sorted\n");
        return 0;
    }
}

int key;

scanf("%d",&key);

int low = 0, high = number_of_elements-1, mid = (low + high)/2;

while(low <= high)
{
    if(array[mid] < key)
    {
        low = mid + 1;
    }

    else if(array[mid] == key)

```

```
{  
    printf("%d found at location %d.\n", key, mid+1);  
    break;  
}  
else(array[mid] > key)  
{  
    high = mid-1;  
}  
mid = (low + high)/2;  
}  
if(low>high)  
{  
    printf("Element not found\n");  
}  
return 0;  
}
```